

Cloud Computing

Mini Project Assignment 1

Muhammed Emin Haliş
20250821

Project Title: The Tarnished Vault - A Distributed Game loadout API

1. Concept and Overview

1.1. Concept Overview:

The proposed application is a centralized web platform designed for communities of complex action and RPG titles, such as Elden Ring, Warframe, and Helldivers 2. In modern gaming, communities heavily emphasize both mechanical character optimization (stat 'builds') and visual customization ('fashion'). However, players often rely on scattered forum posts or social media to share these loadouts. This project addresses that gap by providing a dedicated, structured environment. The platform allows users to upload, store, and review comprehensive character profiles. By natively supporting both granular, text-based statistical data and high-resolution screenshots, the application creates a unified gallery experience where players can discover, share, and review community creations.

1.2. Core Functionality:

The core functionality of the application resolves around a streamlined Create and Read (CRUD) workflow, divided into the following system actions:

- **Build Submission:** Users submit a loadout via an HTTP POST request. This payload contains structured textual data (e.g, weapon types, distributions) alongside a single binary file representing a screenshot of the character.
- **Data Routing & Persistence:** Upon receiving the request, the backend API explicitly routes the data based on its type. The structured text is serialized into a JSON document and persisted in generating a direct access URL that is saved within the CosmosDB document to link the two data points
- **Asynchronous processing:** The upload of an image to the Blob Storage automatically triggers a serverless Azure Function. This function operates asynchronously in the background to log the file size and timestamp, demonstrating event-driven architecture without interrupting the user's primary web request.
- **Build Retrieval:** Users can request to view the community gallery. The API retrieves the NoSQL documents from CosmosDB using the embedded Blob URLs to serve both the statistical data and the visual screenshot seamlessly to the client

2. Project Objectives

2.1 Primary Goals:

The primary objective of this project is to architect and deploy a highly scalable, distributed backend system from first principles. Rather than relying on a monolithic structure, the goal is to build a decoupled web application that effectively utilizes cloud-native paradigms to handle distinct data types efficiently.

2.2 Technical Scope:

The technical scope focuses on integrating distinct Microsoft Azure services to fulfill specific architectural roles. This includes cleanly separating structured NoSQL document storage from heavy binary BLOB storage. Furthermore, the project aims to implement event-driven serverless computing to handle background tasks asynchronously, and to enforce “Infrastructure as Code” (IaC) principles by fully automating the provisioning of the cloud environment via scripting

3. Proposed Cloud Infrastructure and Azure Services:

To fulfill the architectural goals and the project requirements, the backend will be constructed using the following Microsoft Azure services, integrated to form a cohesive, distributed system:

3.1 Compute & Hosting: Azure App Service and Docker:

The core application logic will be developed as lightweight C# .NET minimal API. To ensure environmental consistency, this application will be packaged inside a Docker Container. The containerized application will be hosted on Azure App Service, providing a scalable compute environment. Furthermore, the source code will be maintained in a GitHub repository, which will be directly integrated with Azure App Service to establish a continuous deployment pipeline.

3.2 Data Persistence: Azure CosmosDB:

Given the highly variable nature of character stats across different games (e.g, varying attributes, armor slots, and weapon types), a rigid relational database is suboptimal. Therefore, the application will utilize the NoSQL CosmosDB database to store user submissions as flexible JSON documents, ensuring high read/write performance for the statistical data

3.3 Binary Storage: Azure Blob Storage:

While the text-based stat submissions are the core of the application, users have the option to enhance their build profiles with visual context. To support this without causing database bloat, the application utilizes a dedicated cloud storage, specifically Azure BLOB storage, to securely hold any optional high-resolution character screenshots uploaded by users.

3.4 Event-Driven Automation: Azure Functions:

To handle background processing asynchronously, the architecture incorporates serverless computing via an Azure Function. This function is configured with a Blob Trigger; in instances where a user opts to upload a screenshot, the successful write to the BLOB storage automatically triggers the function. It executes to log the system event (such as file size and upload timestamp without consuming the main App Service's) compute resources

3.5 Infrastructure as Code (IaC):

To ensure the environment is reproducible and to eliminate manual configuration errors, The project's cloud infrastructure will be created automatically using a script. A centralized Bash script utilizing the Azure CLI has been developed to sequentially provision the Resource Group, App Service Plan, CosmosDB account, Storage Account, and Function app

4. Project Management And Work Plan

4.1 Project Roles and Responsibilities:

To ensure efficient development within the available timeframe the workload has been structured into distinct engineering phases:

- **Infrastructure & CI/CD:** Responsible for writing the Azure CLIW automation script, configuring the Docker container, and setting up the GitHub Actions deployment pipeline.
- **Core API & Routing:** Responsible for developing the C# .NET Minimal API, handling HTTP request logic, and routing data correctly between cloud services
- **Data & Event Processing:** Responsible for structuring the CosmosDB NoSQL JSON data models, integrating Azure Blob Storage for optional image uploads, and developing the Azure Function for background event logging.
- **Documentation & Analysis:** Responsible for generating system architecture reports, calculating theoretical cloud costs, and compiling the final theoretical study.

4.2 Project Timeline:

Cloud Computing Mini Project Assignment

Muhammed Emin Haliş
20250821

Task List April 2 April 8 April 9 April 14 April 15 April 24 April 25 May 3 May 4 May 8 May 9 May 14 May 15 May 18 May 19 May 21

